

Programming Assignment 3

Neural Networks

1. Assignment Structure

DISCLAIMER: Students are reminded that this assignment is an individual effort, and each participant is expected to submit their own work. Plagiarism is strictly prohibited, including the utilization of chatbots or any other automated systems. Be mindful of academic integrity and ensure that your submission is an authentic representation of your understanding and effort.

2. Neural Networks [100 Marks]

In this section, you will implement key components of a neural network, including the forward pass, backward pass, and various activation functions. I've provided a YouTube playlist link in the notebook by educator 3blue1brown for anyone interested in getting an intuitive understanding of neural networks. In this part, you will implement the following inside the Neural Network class:

2.1 Forward Pass

The forward pass is the first operation in a neural network where the input data is transformed through each layer to produce the final prediction. In this programming assignment, you'll be implementing a flexible forward pass that can handle any number of layers in a neural network.

Mathematical Formulation: For a neural network with L layers, the forward pass can be expressed as follows:

$$\text{output}^{(l)} = \text{activation}^{(l)}(\text{output}^{(l-1)} \cdot W^{(l)} + b^{(l)})$$

Here:

- $\text{output}^{(l)}$ is the output of layer l .
- $W^{(l)}$ is the weight matrix for layer l .
- $b^{(l)}$ is the bias vector for layer l .
- $\text{activation}^{(l)}$ is the chosen activation function for layer l (e.g., sigmoid, ReLU, Leaky ReLU).

For the final layer in classification tasks, use the softmax activation function:

$$\text{Softmax}(x_i) = e^{(x_i)} / \sum e^{x_j}$$

Here:

- x_i is the input to the softmax function for class i .
- C is the total number of classes.

Your task is to implement this generalized forward pass, including the softmax activation function for the final layer in classification tasks, allowing for flexibility in the number of layers in the neural network. Note that for regression tasks, we don't use any activation function in the last layer.

2.2 Backward Pass

The backward pass involves the calculation of gradients for parameters (weights and biases) with respect to the loss. This process is particularly complex in multilayered networks (such as the ones you will be implementing), where the chain rule is applied iteratively to efficiently propagate the error backward through the layers.

Mathematical Formulation:

Let's consider a neural network with one hidden layer.

For the weights (W), the gradient is calculated using the chain rule iteratively:

$$\frac{\partial \text{Loss}}{\partial W} = X^T \cdot \frac{\partial \text{Loss}}{\partial \text{output}} \cdot \frac{\partial \text{output}}{\partial \text{input}} \cdot \frac{\partial \text{input}}{\partial \text{weights}}$$

For the biases (b), the gradient is calculated similarly:

$$\frac{\partial \text{Loss}}{\partial b} = \frac{\partial \text{Loss}}{\partial \text{output}_i} \cdot \frac{\partial \text{output}_i}{\partial \text{input}_i} \cdot \frac{\partial \text{input}_i}{\partial b_i}$$

For a more detailed explanation, please refer to the lecture slides. You will be implementing this backward pass algorithm, and it should work for any arbitrarily sized Neural Network.

2.3 Weight Update

This process involves adjusting the weights based on the calculated gradients obtained during the backward pass.

Mathematical Formulation

For each layer in the neural network, the weight update can be expressed as follows:

For the weights (W):

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial \text{Loss}}{\partial W_{\text{old}}}$$

For the biases (b):

$$b_{\text{new}} = b_{\text{old}} - \eta \cdot \frac{\partial \text{Loss}}{\partial b_{\text{old}}}$$

Where:

- W_{new} and b_{new} are the updated weights and biases, respectively.
- W_{old} and b_{old} are the current weights and biases.
- η is the learning rate.
- $\partial \text{Loss} / \partial w_{\text{old}}$ and $\partial \text{Loss} / \partial b_{\text{old}}$ are the gradients of the loss with respect to the weights and bias

Implement this weight update step in your neural network class. Instructions are in the notebook.

2.4 Stochastic Gradient Descent (SGD) - One Example at a Time

Full-Batch Gradient Descent has been implemented for you. Implementing SGD involves updating the parameters for each training example (by looping over the entire dataset).

2.5 Mini-batch Stochastic Gradient Descent (Mini-batch SGD)

Extend the SGD implementation to handle mini-batches of training examples. Adjust the weight and bias updates accordingly based on the mini-batch gradients.

2.6 Activation Functions

Implement the following activation functions (as well as their forward & backward pass algorithms in the `forward_pass` and `backward_pass` functions):

Sigmoid Activation Function

$$\text{Sigmoid} = 1 / (1 + e^{-x})$$

ReLU (Rectified Linear Unit) Activation Function

$$\text{ReLU}(x) = \max(0, x)$$

Leaky ReLU Activation Function (with fixed leaky slope at 0.05)

The Leaky ReLU activation function introduces a small slope for negative input values, preventing complete suppression. It was designed to tackle the dying ReLU problem. The mathematical expression for the Leaky ReLU function is defined as follows:

$$\text{Leaky ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ 0.5x & \text{if } x \leq 0 \end{cases}$$

It's recommended that you test your code on small examples to verify correctness.

2.7 Tasks for PA1

Task 1: Implementation (70 marks)

Implement all the components described above, including the forward pass, backward pass, SGD, Mini-batch SGD, and activation functions (Sigmoid, ReLU, Leaky ReLU).

Task 2: Classification Experimentation (15 Marks)

Experiment with different network sizes, activation functions, learning rates, epochs, and optimizers for the moons dataset. Comment on which weight activation function + optimizer combinations yield the best results. Conduct at least 5 experiments with various nodes per layer and the number of layers, calculating the total number of parameters for each model (add these as comments next to your network_size declaration).

Task 3: Regression Experimentation [15 Marks]

Use the model created for a regression task. Sample from the regression equation:

$$y = 2X_1^5 + 4X_2^2 - X_1X_2 + 10X_2 \cdot (\sin(X_1)^{13.5}) + 3X_2 + 5 + \epsilon$$

where ϵ represents the noise added to the data. Experiment with different sizes and various combinations of optimizers and activation functions. Remember to use a very small learning rate with ReLU. Show results for at least three models.

Good Luck.